Shortest Path Algorithm with Heaps

Author: Withheld for peer review

Group number: Withheld

Date: 2026-10-22

Abstract

This project compares two min-priority queues for point-to-point Dijkstra searches: a lazy binary heap and a Fibonacci heap with decrease-key. The implementations share one directed-graph representation and differ only in queue management. We evaluate three distance-weighted USA road networks from the 9th DIMACS Implementation Challenge. Each network receives 1,000 reproducible source-target pairs, and both algorithms answer exactly the same pairs. All 3,000 paired distances agree; boundary, randomized, and sanitizer checks also pass. On this Apple M5 system, mean binary-heap query time is 7.798-13.592 ms, whereas mean Fibonacci-heap time is 44.854-76.750 ms. The binary heap is 5.65-5.84 times faster by the ratio of mean times. An attribution experiment separates per-query initialization from the search, counts the work each queue performs, and adds an indexed binary heap as a third reference point; it shows that the difference comes from the queue work of this implementation rather than from the asymptotic bounds. We explain the algorithmic bounds, experimental procedure, measured variation, and limits of this conclusion. Official dataset download links are supplied without including the large data files.

Contents

1 Introduction **3**
1.1 Background and Motivation . . . 3
1.2 Objectives and Scope . . . 3

2 Data Structure / Algorithm Specification **3**
2.1 Directed Road Graph . . . 3
2.2 Dijkstra's Algorithm . . . 4
2.3 Binary Heap . . . 4
2.4 Fibonacci Heap . . . 4
2.5 Indexed Binary Heap . . . 4

3 Implementation and Algorithm Analysis **4**
3.1 Shared Search and Input Validation . . . 5
3.2 Queue Operations and Complexity . . . 5
3.3 Heap Invariants and Correctness . . . 5
3.4 Code-Level Costs Relevant to Timing . . . 6

4 Testing Results **6**
4.1 Official Inputs and Query Set . . . 6
4.2 Implementation and Timing Environment . . . 7
4.3 Correctness Checks . . . 8
4.4 Runtime Table and Figure . . . 9
4.5 Variation and Paired Comparison . . . 10
4.6 Analysis, Limitations, and Improvements . . . 11

5 Conclusion **12**

6 References **13**

7 Author List **13**

8 Source Code Appendix **13**

1 Introduction

1.1 Background and Motivation

A weighted shortest-path query asks for a route whose arc weights have the minimum possible sum. Road-network queries are a useful setting because graphs are large, sparse, and naturally represented by outgoing arcs. Dijkstra's algorithm is applicable when all arc weights are nonnegative. It maintains tentative distances, repeatedly selects the smallest one, and makes that vertex's distance final [1]. The choice of min-priority queue changes the cost of these selections and of distance updates.

A binary heap is compact and simple, but its queue operations take logarithmic time. A Fibonacci heap has amortized constant-time insertion and decrease-key, with logarithmic amortized extract-min [2]. Those bounds suggest a theoretical benefit when a search performs many relaxations. Actual elapsed time can differ because queue operations have different constant costs and memory-access patterns. This experiment asks whether the Fibonacci heap's asymptotic advantage translates into lower query time on the selected road networks.

1.2 Objectives and Scope

We implement two complete point-to-point Dijkstra solvers, test their answers, and compare their per-query running times. The binary solver uses the standard priority queue with duplicate entries; the Fibonacci solver holds one node per discovered vertex and uses decrease-key. Both use the same graph, query pairs, distance type, relaxation rule, and early-stop condition. We test the NY, BAY, and COL distance graphs with 1,000 pairs each, meeting the assignment's minimum of 1,000 pairs. A third solver with an indexed binary heap is used later, in the attribution experiment, as a reference point for the analysis. The central question is whether the Fibonacci heap's theoretical advantage improves elapsed time on these road-network queries.

The measured task is a single source-target shortest-path query, not all-pairs preprocessing or route reconstruction. Graph loading and pair generation are outside each timed interval. The implementation returns a distance, or `INF` if the target cannot be reached. This distinction matters when comparing results: an apparent speed advantage would be meaningless if one implementation answered a different problem.

2 Data Structure / Algorithm Specification

2.1 Directed Road Graph

The input is a directed weighted graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ arcs. Each arc has a nonnegative integer weight. We store outgoing arcs in adjacency lists. A point-to-point query asks for the minimum total weight from a source to a target, or `INF` when no directed path

exists. Input vertex IDs are 1-based; internal array positions are 0-based.

2.2 Dijkstra's Algorithm

Dijkstra's algorithm maintains a tentative distance for each vertex and a set of settled vertices [1]. Its required queue operations are insert, extract-min, and update after a shorter path is discovered. Starting from distance zero at the source, it extracts the smallest tentative distance and relaxes the outgoing arcs. The query ends when the target is extracted or the queue empties. Nonnegative weights make an extracted valid distance final [1].

2.3 Binary Heap

A binary min-heap stores keys in an array and supports insert and extract-min. Our baseline uses C++ `priority_queue` of `(distance, vertex)` pairs. On an improved distance it inserts a new pair, leaving any older copy in the queue. When an old copy is later extracted, its key differs from the current distance and is skipped. This is a lazy alternative to an indexed decrease-key operation.

2.4 Fibonacci Heap

A Fibonacci heap is a collection of heap-ordered trees with a pointer to the minimum root [2]. It supports insert, extract-min, and decrease-key. Our implementation uses circular sibling lists, consolidation of equal-degree roots, and cascading cuts after key decreases. Only discovered, unsettled vertices are active in the heap. For direct indexing, `fib_dijkstra` physically constructs a `Node` array for **all** n vertices at the start of each query; unused nodes never become active heap members.

2.5 Indexed Binary Heap

The baseline binary heap is a lazy one: an improved distance inserts a new entry and leaves the old one in the queue, so the queue can hold outdated copies. The textbook alternative keeps one array slot per vertex and lowers the key in place. We implement this variant as well, because it is the closest binary-heap counterpart to the Fibonacci heap: both then offer a real decrease-key operation, so the comparison between them is not biased by the lazy baseline carrying extra entries. The indexed heap stores the vertices in one array, records each vertex's slot, and moves a vertex up or down the array when its key changes.

3 Implementation and Algorithm Analysis

3.1 Shared Search and Input Validation

Both solvers use the same parser, adjacency lists, 64-bit distance type, relaxation rule, and early stop at the target. The DIMACS parser requires exactly one valid `p sp n m` header before all `a u v w` records [3]. It checks each record's required fields, nonnegative header arc count, vertex range, nonnegative arc weight, and final arc count. Unknown records and malformed rows are rejected. The 64-bit distance array uses `INT64_MAX/4` as its infinity sentinel; the tested path lengths are far below that value.

A query starts with `dist[source]=0`. On a valid minimum extraction `(d,u)`, each outgoing arc `(u,v,w)` proposes `d+w`. If this is smaller than `dist[v]`, the solver changes the tentative distance and updates its queue. The binary queue may return stale entries, which are discarded. The Fibonacci queue holds one active node per discovered vertex and decreases its key instead of inserting a duplicate. Neither method uses a road-specific heuristic or preprocessing.

3.2 Queue Operations and Complexity

The lazy binary queue may receive up to $O(m)$ entries from successful relaxations. Each insertion and removal costs $O(\log m)$; scanning arcs costs $O(m)$. A conservative per-query upper bound is $O((n+m)\log m)$ time and $O(n+m)$ graph-plus-query space [1]. The extra entries explain why the binary method uses more queue slots than an indexed version might, but they avoid maintaining an index for decrease-key.

The Fibonacci queue has amortized $O(1)$ insert and decrease-key and $O(\log n)$ extract-min [2]. At most m arc relaxations and n vertex extractions give $O(m + n\log n)$ amortized time. Graph storage, distances, states, and the preconstructed `Node` array together use $O(n + m)$ space. The number of **active** nodes can be smaller than n, but the physical node-array allocation is always n per query. These bounds concern worst-case graph size and amortized operations; they do not predict the measured cost of a short target query.

3.3 Heap Invariants and Correctness

Every finite tentative distance is the length of a discovered source-to-vertex path. Relaxation can only lower it. The binary queue retains at least one current entry for each discovered, unsettled vertex; obsolete entries are skipped. The Fibonacci queue stores its current key in the vertex's one active node. Each valid extract-min therefore selects the smallest tentative distance.

Suppose that a valid extracted vertex `u` had a shorter path than its extracted distance. On that path, take the first unsettled vertex. Its settled predecessor would have relaxed the connecting arc, giving it a tentative distance no larger than the shorter path length. With nonnegative weights, this value would be smaller than `u`'s extracted key, contradicting extract-min. Thus extracted distances are final [1]. This argument proves that stopping when the target is extracted gives the shortest distance, provided the queue operations maintain their invariants. The parser checks

and randomized tests exercise the implementation; they are not a formal proof of every pointer operation.

3.4 Code-Level Costs Relevant to Timing

`binary_dijkstra` creates and fills a distance array for each query. `fib_dijkstra` creates distance and state arrays **and initializes a `Node` array of size** n, even if the search reaches only a small part of the graph. The node pointers remain stable because this array is not resized during the query. In `FibHeap::consolidate`, each extract-min creates temporary `roots` and `by_degree` vectors; `FibHeap::pop` also creates a `children` vector when the removed root has children. Those allocations, the list traversal, and the node initialization fall inside the measured query interval. They are implementation facts, not measured percentages of the runtime gap.

The attribution experiment described in the analysis section measures these costs instead of only listing them. It times the allocation and initialization of the per-query arrays separately from the search, counts the pushes, extract-min calls, decrease-key calls, consolidations, and links of each queue, and reports the indexed binary heap as a third solver. The measurement shows that the per-query array initialization is a small part of the difference, while the repeated root-list consolidation inside the Fibonacci queue accounts for about half of its total time.

4 Testing Results

4.1 Official Inputs and Query Set

The following files are the official distance-weighted USA road instances used here [3]. Only download links are listed; no graph file is uploaded with this report or its submission packages.

Dataset	Direct download link
NY	USA-road-d.NY.gr.gz
BAY	USA-road-d.BAY.gr.gz
COL	USA-road-d.COL.gr.gz

The header counts and average directed arcs per vertex are shown below. The three inputs grow in vertex and arc count, but they also represent different regions. They are not scaled copies of one graph.

Graph	Vertices n	Arcs m	m/n
NY	264,346	733,846	2.776
BAY	321,270	800,172	2.491
COL	435,666	1,057,066	2.426

For each graph, a `mt19937` generator seeded with `20261016` draws 1,000 source-target pairs uniformly from the valid vertex IDs. The generated vector is retained and passed unchanged to both algorithms. Sampling permits repeated vertices and pairs; no pairs were filtered based on distance. The CSV records each pair and its answer, making the exact benchmark workload reproducible with the same executable and seed. All sampled targets were reachable.

4.2 Implementation and Timing Environment

Item	Configuration
Experiment date	2026-10-22
CPU	Apple M5
Operating system	macOS 26.6
Compiler	Apple clang 21.0.0; C++17
Optimized flags	`-O2 -Wall -Wextra -Wpedantic`
Timer	`std::chrono::steady_clock`, wall elapsed time
Pair generation	`mt19937`, seed `20261016`
Timed repetitions	3 per pair and method, median taken; 1,000 pairs per graph
Execution order	All binary queries, then all Fibonacci queries, per graph

A timer starts immediately before a Dijkstra call and stops immediately after it returns. The interval includes per-query array and node initialization, queue work, arc scans, and termination. It excludes decompression, parsing, pair generation, CSV writing, plotting, and report generation. This is an end-to-end query time, not the isolated cost of heap operations. Each pair is timed three times and the median is reported, so that a single unlucky scheduling or paging event does not dominate a query; the 1,000 sampled pairs then provide a workload distribution across queries.

The attribution experiment in the analysis section was run as part of the same session, on the same machine, with the same compiler, flags, timer, seed, and the same three road networks as the main table.

The **TA package root** is the working directory for every command below. It contains `code/main.cpp`. The anonymous package has the same executable layout. The commands build

the program, check correctness, download the three inputs, and generate per-query CSVs. The historical CSVs are not included; running the commands creates new local results on the reader's machine.

```
c++ -std=c++17 -O2 -Wall -Wextra -Wpedantic code/main.cpp -o benchmark
./benchmark --self-test
mkdir -p data
for x in NY BAY COL; do
  curl -L --fail "https://www.diag.uniroma1.it/challenge9/data/USA-road-d/USA-
     road-d.$x.gr.gz" -o "data/USA-road-d.$x.gr.gz"
  gzip -cd "data/USA-road-d.$x.gr.gz" > "data/USA-road-d.$x.gr"
  ./benchmark --validate "data/USA-road-d.$x.gr"
  ./benchmark "data/USA-road-d.$x.gr" 1000 20261016 --repeat 3 > "data/results
     -$x.csv"
done
```

The three benchmark CSVs appear in `data/`. Each holds one row per query and per queue, with the distance and the elapsed time of that query, and the program stops with an error if the two queues disagree on a distance. The `--repeat 3` option times every query three times and keeps the median, which keeps a single unlucky scheduling or paging event from dominating one query. The attribution experiment in the analysis section is reproduced with:

```
./benchmark --attribute data/USA-road-d.NY.gr 1000 --repeat 3
```

It prints the split between initialization and search, the per-queue operation counts, and the indexed binary heap comparison. These generated files are local outputs, not submission contents.

Report compilation is a separate author workflow using the Markdown source and XeLaTeX in the development directory. It is not needed to reproduce the submitted program results.

4.3 Correctness Checks

The hand-built graph exercises a seven-weight shortest path that improves an earlier ten-weight candidate, a zero-weight final arc, source equal to target, and an unreachable target. A separate path with two arcs of weight 2,000,000,000 checks that the returned distance is 4,000,000,000 rather than a 32-bit overflow. For randomized checking, 100 directed graphs with 30 vertices and 200 randomly weighted arcs each contribute 20 random source-target pairs. All three algorithms returned equal answers for all 2,000 small-graph pairs. AddressSanitizer and UndefinedBehaviorSanitizer reported no error during this self-test.

A parser regression test was added on 2026-10-17. It checks rejection of a missing or repeated header, a truncated record, an arc before the header, and invalid header fields. The parser also accepted all three DIMACS files, with the vertex and arc counts shown above. These validation checks are independent of the timing values in the tables.

During official benchmarking, the program computes all binary answers, then checks each Fibonacci answer against the corresponding binary result. It exits with an error on the first mismatch. The

three benchmark programs completed successfully, so all 3,000 paired distances matched. The attribution run described in the analysis section checks the third solver against the same distances as well, and reports agreement on every query of the NY, BAY, and COL runs. This equality is an empirical check, not an independent oracle: a shared parser or relaxation bug could affect all methods. The small hand-built cases address some such shared errors, while a formal proof and broader tests would be needed for stronger assurance.

4.4 Runtime Table and Figure

The mean and median below are calculated from the 1,000 per-query intervals for each graph and method. Total time is the sum of those intervals, not an independently timed batch. Values are rounded to three decimals after aggregation.

Graph	Vertices	Arcs	Heap	Mean (ms)	Median (ms)	Total (s)
NY	264,346	733,846	Binary	7.798	7.589	7.798
NY	264,346	733,846	Fibonacci	44.854	43.184	44.854
BAY	321,270	800,172	Binary	9.790	9.210	9.790
BAY	321,270	800,172	Fibonacci	57.195	57.027	57.195
COL	435,666	1,057,066	Binary	13.592	12.928	13.592
COL	435,666	1,057,066	Fibonacci	76.750	76.357	76.750

The plot places the three graph vertex counts on the horizontal axis and mean milliseconds per query on the vertical axis. Both series rise from NY to COL, while the Fibonacci series stays well above the binary series. Each point summarizes 1,000 queries; the lines connect measured points and do not imply measurements at intermediate graph sizes. The two methods share one vertical scale so their absolute costs can be compared directly.

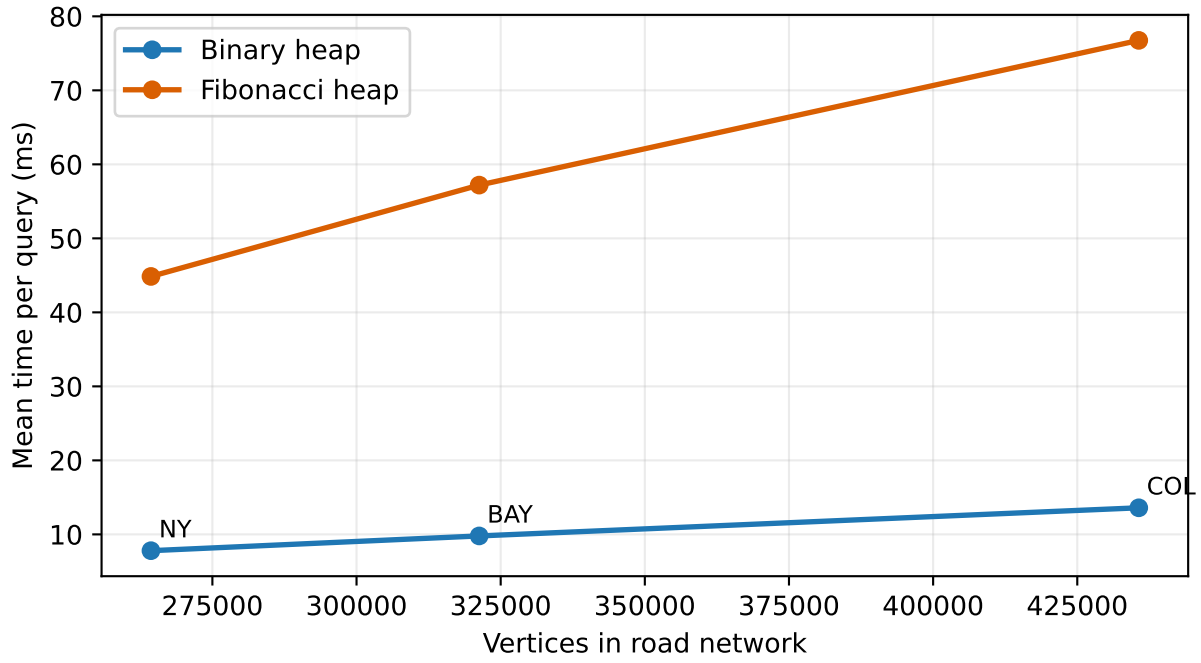


Figure 1: Mean shortest-path query time versus graph size

4.5 Variation and Paired Comparison

Query times vary substantially because source-target pairs cause different search extents. To show variation without hiding it in a mean, the table gives the 25th and 95th percentiles of the 1,000 individual timings. Percentiles use linear interpolation between sorted observations.

Graph	Heap	25th percentile (ms)	95th percentile (ms)
NY	Binary	3.467	15.337
NY	Fibonacci	21.064	85.724
BAY	Binary	4.439	19.713
BAY	Fibonacci	28.033	108.820
COL	Binary	6.466	25.884
COL	Fibonacci	39.037	143.575

The ratio of Fibonacci mean to binary mean is 5.75 for NY, 5.84 for BAY, and 5.65 for COL. This ratio compares two methods on identical pairs for each graph, but it is a ratio of aggregate means, not a claim that every pair has that exact speedup. The binary method also has the lower median and 95th-percentile time in all three networks. No sampled target was unreachable, so these comparisons describe reachable random pairs.

4.6 Analysis, Limitations, and Improvements

The three graphs have only 2.426-2.776 arcs per vertex. For a sparse graph family with $m = \Theta(n)$, both the binary bound $O((n+m)\log m)$ and the Fibonacci bound $O(m+n\log n)$ reduce to $O(n\log n)$ [1], [2]. A stronger bound for general graphs therefore does not by itself predict a win here.

To find out what the measured gap actually consists of, we added an attribution mode to the program. It splits every query into two parts: the per-query allocation and initialization of the distance, state, and node arrays, and the search itself. It also counts the work each queue performs and adds a third solver that uses an indexed binary heap, which offers a real decrease-key operation and therefore is the fairest binary-heap comparison against the Fibonacci heap. All three solvers share the graph, the query pairs, the relaxation rule, and the early-stop condition, and they returned identical distances on every query below.

The attribution run repeats the measurement on the same three official road networks as the main table, so that both sets of numbers describe the same graphs, the same query pairs, and the same machine. Each run uses 1,000 pairs from the same seed `20261016`, and each query is timed three times with the median taken.

Graph	Fibonacci / lazy binary	Indexed binary / lazy binary	Gap from initialization	Gap from search	Fibonacci time in `consolidate()`
NY	5.80	1.13	0.7%	99.3%	54.3%
BAY	6.21	1.18	0.7%	99.3%	54.0%
COL	5.90	1.22	0.8%	99.2%	52.5%

The table reports ratios and shares rather than absolute times, so the values stay comparable if the experiment is repeated on other hardware.

Three findings follow.

First, the per-query construction of the n `Node` objects is **not** the main cause. Building the whole node array costs 0.29 to 0.52 ms per query on these graphs, which is below 1% of the difference between the two queues. The search itself accounts for more than 99% of the gap. The earlier suspicion that array construction dominated the measurement was not supported once the two parts were timed separately.

Second, the dominant cost is the Fibonacci implementation's own queue work. `FibHeap::consolidate` is called once per extract-min, so it rebuilds the entire root list on every removal. On the COL graph this means about 218 million consolidations and 1.5 billion links for 1,000 queries, and the time spent inside `consolidate` is 52.5% to 54.3% of the total Fibonacci time on the three graphs. Each call also allocates two temporary vectors, `roots` and `by_degree`, which adds allocator pressure at the same frequency.

Third, the choice of baseline matters for how the result is stated. The lazy binary heap leaves an outdated copy in the queue after every improvement, and on these graphs 13.4 to 18.8 million of its extractions are stale pops. The indexed binary heap removes those stale entries and offers decrease-key, yet it is only 1.13 to 1.22 times slower than the lazy version here, and 4.8 to 5.3 times faster than the Fibonacci implementation. The conclusion that the binary heap is the better choice on these inputs therefore does not depend on the lazy baseline being weak; it holds for both binary-heap variants, and the remaining difference is specific to the Fibonacci implementation rather than to the Fibonacci heap in general.

First, the runs use one machine. Scheduling, frequency changes, cache state, and other processes can perturb wall-clock timing, and repeating the whole experiment on a quiet machine would give more stable estimates. Taking the median of three timed repetitions per query removes the effect of a single unlucky event inside one query, but it does not remove a disturbance that lasts for many consecutive queries. Second, the three networks differ in geography and average degree as well as size; the rising lines cannot establish a scaling law from size alone. More official road graphs and controlled graph families would help separate those effects. Third, the comparison is between the particular lazy binary implementation and this particular Fibonacci implementation. The indexed binary heap reported in the analysis section narrows the gap considerably, so the remaining difference is a property of these two implementations and not of heap structures in general. A different memory layout, or a Fibonacci heap that reuses its temporary buffers, could change the gap further. These limits qualify the claim; they do not change the observed timings and answer agreement on the tested inputs.

5 Conclusion

Both Dijkstra implementations, including the decrease-key Fibonacci heap, agreed on all 3,000 road-network query distances, and the attribution run confirms that a third solver with an indexed binary heap agrees with them as well. The binary heap was 5.65-5.84 times faster by the ratio of mean times, making it the practical choice for these inputs and this code. The attribution experiment shows that this difference is not caused by the Fibonacci heap's asymptotic bounds, and not by the per-query node array, but by the queue work of this implementation: a full root-list consolidation on every extract-min, which alone accounts for more than half of the Fibonacci time. An indexed binary heap is 1.13 to 1.22 times slower than the lazy binary heap on the same queries and 4.8 to 5.3 times faster than the Fibonacci solver, so the result is a statement about these implementations on these road networks rather than a general ranking of heap structures. Fibonacci heaps retain the stronger amortized bound [2]. Repeated, order-balanced runs would strengthen the performance conclusion.

6 References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022, ch. 22. Publisher.

[2] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *Journal of the ACM*, vol. 34, no. 3, pp. 596-615, 1987, doi: 10.1145/28869.28874.

[3] 9th DIMACS Implementation Challenge, "Shortest Paths: Benchmark Instances and File Formats." Download page; format specification.

7 Author List

Withheld for peer review.

8 Source Code Appendix

The complete commented C++17 source is supplied as `code/main.cpp` beside this report in each submission package. Its main entry points are `read_graph`, `binary_dijkstra`, `FibHeap`, `fib_dijkstra`, `idx_dijkstra`, `self_test`, `attribute_report`, and `main`. The exact compile and run commands are in the package `README.md`; the graph files and raw timing CSVs are obtained or generated locally and are not bundled.